

Guía del Demo de Hilos

Cómo leer el código de *demo-hilos* antes de empezar la Práctica 2 (Quetzal Space Defender)

Este documento explica el proyecto *demo-hilos* que se les entrega junto con el enunciado de la práctica. El demo es un ejemplo funcional, no la solución de la práctica: les muestra **cómo usar hilos en Java sin romper nada y cómo organizar el código en Modelo / Vista / Controlador**, pero deja fuera casi todo lo que la práctica realmente evalúa (pilotos, menú, power-ups, persistencia, reportes con datos reales).

Usen esta guía así:

- Primero corran el demo y jueguen con él un rato (flechas de dirección, botón "Generar reporte").
- Después lean esta guía de principio a fin, con el código abierto al lado.
- Al final hay una lista de lo que el demo **no** resuelve: esa lista es, en la práctica, su plan de trabajo.

Regla de oro: copiar clases de este demo tal cual (Movil, Escena, Arbitro, Teclado, etc.) está permitido y es el punto del ejercicio. Lo que no deben copiar es la idea de que ya está resuelto: las reglas del juego (naves, puntajes reales, power-ups, menú) las tienen que diseñar ustedes.

1. Qué es este demo (y qué no es)

Sí incluye: una ventana Swing donde varios objetos se mueven solos al mismo tiempo (cada uno en su propio hilo), un jugador que se controla con las flechas, un "árbitro" que revisa colisiones, generación de enemigos y proyectiles en hilos separados, y un botón que genera un reporte en HTML con una imagen embebida.

No incluye (y es justo lo que la práctica pide construir): pantalla de menú, creación/selección de piloto y modelo de nave, las tres mecánicas de premios/obstáculos (Snitch, Asteroide/Bludger, Contenedor Quaffle), persistencia real de pilotos e historial, gráficas con JFreeChart, y las reglas concretas de puntaje. Donde el código deja algo así a propósito, va a estar comentado con ESTO ES SOLO UN EJEMPLO o similar.

Cómo ejecutarlo

Desde la carpeta del proyecto:

```
macOS / Linux:
./compilar-y-correr.sh

Windows:
compilar-y-correr.bat
```

Ambos scripts compilan `src/demo/**/*.java` hacia una carpeta `out/` y ejecutan `demo.controller.Main`. En la ventana: flechas de dirección mueven la nave, y el botón "Generar reporte" crea `reportes/top.html + top.png` y los abre en el navegador.

2. Por qué está dividido en model / view / controller

El código de `src/demo` está separado en tres carpetas (paquetes de Java). La idea de fondo: cada clase debería poder responder "¿de qué otra clase dependo?" con una sola capa, nunca con las tres.

Carpeta	Responde a la pregunta...	Clases
model/	¿Qué existe en el juego y cómo se comporta, sin saber nada de pantalla ni teclado?	Config, Movil, Jugador, Enemigo, Proyectil, Escena, EntradaJugador
view/	¿Cómo se le muestra al usuario lo que hay en el modelo (en pantalla o en un archivo)?	PanelEscena, Reporte
controller/	¿Quién conecta el tiempo (hilos) y la entrada del usuario con el modelo?	Main, Arbitro, Teclado, HiloRender, HiloSpawn, HiloDisparo

La regla que no se puede romper: el modelo no puede depender de la vista ni del controlador. Puede sonar abstracto, así que aquí hay un ejemplo real de por qué importa, tomado de este mismo código.

Al mover las clases a sus carpetas apareció un problema: `Jugador` (modelo) necesitaba leer las flechas presionadas, y la forma obvia era recibir un objeto `TecLado` — pero `TecLado` es del controlador (usa clases de AWT para escuchar el teclado). Si `Jugador` dependiera directamente de `TecLado`, el modelo dependería del controlador: dirección incorrecta.

La solución: una interfaz que vive en el modelo y que el controlador implementa.

```
// model/EntradaJugador.java (interfaz, vive en el MODELO)
public interface EntradaJugador {
    boolean arriba();
    boolean abajo();
    boolean izquierda();
    boolean derecha();
}

// model/Jugador.java -> depende solo de la interfaz
public Jugador(EntradaJugador entrada, int pausaMs) { ... }

// controller/Teclado.java -> IMPLEMENTA la interfaz del modelo
public class Teclado extends KeyAdapter implements EntradaJugador { ... }
```

Con esto, Jugador solo sabe que existe "algo que puede decirme si arriba/abajo/izquierda/derecha están presionados" — no sabe ni le importa que ese algo sea un `KeyListener` de Swing. Cuando ustedes decidan cómo van a manejar el menú, los pilotos o la selección de nave, pregúntense lo mismo: *¿esta clase de modelo realmente necesita saber que existe una ventana, o solo necesita que alguien le pase datos?*

Vista leyendo Controlador: una simplificación consciente

`PanelEscena` (vista) lee directamente el puntaje de `Arbitro` (controlador) para dibujarlo en pantalla. En un MVC más estricto, el puntaje viviría en el modelo y el controlador solo lo actualizaría; aquí se simplificó para no meter una clase extra solo para guardar un número. Es una decisión de diseño válida para un programa de este tamaño — mencionarla en su Manual Técnico si hacen lo mismo demuestra que fue una decisión, no un descuido.

3. Conceptos de hilos que aparecen en el código

Todos estos conceptos ya están usados y comentados en el código; aquí se explican juntos para que tengan el panorama completo antes de leer clase por clase.

3.1 Cada Movil es su propio hilo

Movil (model/Movil.java) es una clase abstracta que hereda de Thread. Cada nave, enemigo o proyectil ES un hilo: tiene su propio `run()` que se mueve solo, duerme un poco, y se repite. Nadie más mueve a un Movil; él se mueve a sí mismo.

```
public abstract class Movil extends Thread {
    protected volatile int x, y;
    protected volatile boolean vivo = true;
    ...
    public void run() {
        while (vivo) {
            x += velocidadX;
            if (x + ancho < 0 || x > Config.ANCHO) vivo = false;
            try { Thread.sleep(pausaMs); }
            catch (InterruptedException e) { vivo = false; }
        }
    }
}
```

Un objeto extiende Thread se vuelve un hilo, pero no se ejecuta solo: hay que llamar a `.start()` (nunca directamente a `.run()`), eso solo ejecutaría el método como si fuera una llamada normal, en el mismo hilo que lo llamó).

3.2 volatile — visibilidad entre hilos

Varias variables (`x`, `y`, `vivo` en Movil.java:15-17; los indicadores de Teclado; `bloqueadoHasta` en Jugador) son `volatile`. Cada una la escribe un hilo y la lee otro distinto (por ejemplo, el hilo del Movil escribe `x`, y el hilo que pinta lo lee). Sin `volatile`, el hilo lector puede quedarse viendo un valor viejo guardado en su caché — el objeto se movería "de verdad" pero en pantalla se vería congelado o con saltos raros.

3.3 synchronized — evitar condiciones de carrera

Escena (model/Escena.java) es el arreglo compartido: es el único lugar donde todos los hilos se tocan entre sí (agregan y quitan móviles al mismo tiempo). Por eso **todos** sus métodos son `synchronized`: solo un hilo a la vez puede estar dentro de `agregar()`, `eliminar()`, etc. Si no fuera así, dos hilos podrían intentar agregar al mismo tiempo y uno pisaría el trabajo del otro.

Regla práctica: si una variable la puede escribir *más de un hilo* (no solo leerla), necesita `synchronized` (o algo equivalente), no le basta con `volatile`.

3.4 Copia defensiva ("instantánea")

`Escena.instantanea()` devuelve una **copia** del arreglo interno, no el arreglo original. Así, quien la recibe (el árbitro revisando colisiones, la vista dibujando) puede recorrerla tranquilo aunque en ese mismo instante otro hilo esté agregando o quitando elementos del arreglo real. Es el mismo patrón en `Arbitro.run()` y en `PanelEscena.paintComponent()`.

3.5 Hilos daemon

Casi todas las clases-hilo llaman `setDaemon(true)` en su constructor (`Movil.java:31`, `Arbitro.java:21`, `HiloRender.java:13`, `HiloSpawn.java:18`, `HiloDisparo.java:19`). Un hilo daemon no le impide a Java cerrar el programa cuando se cierra la ventana. Si se les olvida marcar algún hilo como daemon, pueden cerrar la ventana y el proceso se queda vivo en segundo plano (tienen que matarlo a mano).

3.6 Un solo árbitro, no "cada quién revisa lo suyo"

`Arbitro` es el único hilo que revisa colisiones, comparando cada par de móviles vivos con una prueba de caja contra caja (AABB, cuatro comparaciones — `Arbitro.chocan()`). Si en cambio cada proyectil revisara colisiones por su cuenta, dos proyectiles podrían "matar" al mismo enemigo en el mismo instante y contar puntos dos veces. Centralizar la decisión en un solo hilo evita ese tipo de condición de carrera sin necesitar más `synchronized` del que ya hay.

`Arbitro.resolve(a, b)` es intencionalmente mínimo: si el jugador está involucrado no hace nada, y para cualquier otro par solo mata a ambos y suma 1. Esa es la parte que ustedes deben reemplazar con las reglas reales (Snitch, Bludger, Quaffle, puntajes según su enunciado). El patrón (un hilo, AABB, un método `resolve()` donde se decide) sí lo pueden reutilizar tal cual.

4. Recorrido por cada clase

Tabla de referencia rápida. "Reutilizable tal cual" quiere decir que la técnica que muestra la clase aplica directamente a la práctica; no quiere decir que baste con copiarla sin adaptar nombres ni campos a su propio diseño.

Clase	Capa	Qué hace	¿Reutilizable tal cual?
Config	model	Constantes del mundo (ancho/alto de pantalla, pausas por dificultad). Un solo lugar para tocar números.	Sí — así deberían guardar las constantes de sus 3 naves.
Movil	model	Clase abstracta + Thread. Todo lo que se mueve solo hereda de aquí.	Sí — es la base para sus naves/enemigos/proyectiles/power-ups.
Jugador	model	Extiende Movil pero sobrescribe run(): en vez de moverse solo, lee EntradaJugador para saber hacia dónde moverse. También modela un "bloqueo" temporal con una marca de tiempo (bloquear()/estaBloqueado()), sin necesitar un hilo extra que duerma.	El patrón de bloqueo con timestamp sí; a quién y cuándo se le llama bloquear() es su regla de negocio (por ejemplo, Blutger).
Enemigo / Proyectil	model	Subclases muy delgadas de Movil: solo definen tamaño, velocidad y color de partida.	Sí, como plantilla — ustedes necesitan más tipos (Snitch, Asteroide, Quaffle) siguiendo el mismo patrón.
EntradaJugador	model	Interfaz que separa a Jugador de Teclado (ver sección 2).	Sí, como técnica de diseño (el modelo no depende de la UI).
Escena	model	El arreglo compartido de móviles vivos. Vector nativo que duplica su capacidad al llenarse, sin ArrayList. Todos sus métodos son synchronized.	Sí — mismo patrón que necesitan para guardar pilotos e historial de partidas.
Teclado	controller	KeyListener que solo prende/apaga banderas (implementa EntradaJugador). No mueve nada directamente: quien mueve es el hilo de Jugador leyéndolas.	Sí tal cual, para el control de la nave.
Arbitro	controller	Hilo único que revisa colisiones con AABB y decide qué pasa (resolver()). Ver sección 3.6.	El patrón sí; las reglas de resolver() NO — esas son su tarea.
HiloRender	controller	Hilo único que llama panel.repaint() a ~60 fps. Nadie más pide repintar.	Sí tal cual.
HiloSpawn	controller	Genera enemigos nuevos cada cierto tiempo en el borde derecho.	El patrón sí; la variedad de qué generar (enemigos, power-ups, con qué probabilidad) es su diseño.
HiloDisparo	controller	Dispara un proyectil cada cierta cadencia, salvo que el jugador esté bloqueado.	Sí, adaptando la cadencia según la nave elegida (Fácil/Normal/Difícil).

Clase	Capa	Qué hace	¿Reutilizable tal cual?
Main	controller	Arma la ventana Swing, conecta todas las piezas y arranca los hilos en orden. También tiene el apagado limpio (terminarPartida()).	El patrón de "construir y arrancar" sí; ustedes van a tener más pantallas (menú, crear piloto, top) que coordinar.
PanelEscena	view	Único lugar que dibuja: toma una instantánea de Escena y pinta cada móvil como un rectángulo de su color.	Sí tal cual como base de su render.
Reporte	view	Genera un PNG (gráfica de barras dibujada a mano) y un HTML que lo referencia, usando solo FileWriter/PrintWriter/ImageIO (java.io + java.awt). Se imprime a PDF desde el navegador (Ctrl+P).	La técnica de exportar (HTML + imagen + imprimir a PDF) sí. La práctica pide JFreeChart en vez de dibujar las barras a mano, y datos reales en vez de los arreglos quemados que usa Main.

5. Lo que el demo NO resuelve (su lista de trabajo real)

Esta sección compara el demo contra la rúbrica de la práctica. Si algo está en esta lista, no lo van a encontrar hecho en ningún archivo del demo.

Falta en el demo	Detalle
Menú principal	Pantallas para Jugar, Crear Piloto, Top de Puntajes y Salir, con navegación entre ellas. El demo abre la ventana del juego directamente.
Piloto y naves	Clase Piloto con validaciones (sin duplicados/datos inconsistentes) y selección de modelo de nave (Explorador / Caza Estelar / Acorazado), cada una con sus propios tiempos de movimiento y disparo.
Power-ups y obstáculos	Snitch Espacial (150 pts + mata a todos los enemigos visibles), Asteroide/Bludger (ralentiza o bloquea 2s), Contenedor Quaffle (+10 pts). Arbitro.resolver() deliberadamente no las implementa.
Persistencia real	Vectores con el historial de partidas y de pilotos, usados de verdad (no arreglos quemados como en Main.generarReporte()).
Reportes con datos reales + JFreeChart	Reporte.java muestra la técnica (HTML + imagen + imprimir a PDF) pero con datos de ejemplo y una gráfica dibujada a mano; deben conectar datos reales y usar JFreeChart para generar la imagen.
Documentación	Manual Técnico, Manual de Usuario y Diagrama de Flujo en Markdown. No existen en este repositorio de demo.

Preguntas para revisar que entendieron antes de escribir código propio:

- ¿Por qué Escena necesita synchronized pero Movil.x solo necesita volatile? ¿Cuál es la diferencia entre "una variable que un solo hilo escribe y otros leen" y "una estructura que varios hilos modifican"?
- ¿Qué pasaría si en HiloSpawn se les olvida el Thread.sleep() dentro del while?
- ¿Por qué PanelEscena pide una instantánea en vez de recorrer Escena directamente?
- Si agregan un power-up Snitch como una subclase de Movil, ¿qué necesita Arbitro.resolver() para reconocerlo y darle su efecto especial (matar a todos los enemigos visibles)?
- ¿En qué parte de su propio diseño el modelo terminaría dependiendo de la vista o del controlador, y cómo lo evitarían con una interfaz, igual que EntradaJugador?

Este documento y el código de demo-hilos son material de apoyo del curso. No sustituyen el enunciado oficial de la Práctica 2: ante cualquier diferencia, manda el enunciado.